

Eric M. Truong

☎ (669) 265-9732 | @truongmeric@gmail.com |  LinkedIn | 📍 San Jose, CA

EDUCATION

University of California, Santa Cruz

*Bachelor of Science (B.S.), Computer Engineering
Systems Programming Concentration*

Santa Cruz, CA

Sep 2020 – Jun 2024

TECHNICAL SKILLS

Programming Languages: C (8 years), C++ (5 years), Bash (6 years), Python (1 year)

CI/CD & DevOps: Jenkins, Git, Jira, Confluence, Helix, Test Automation, Hardware-in-the-loop (HIL/HITL), Groovy

Yocto Project: Bitbake, Recipe writing (.bb/.bbappend), U-Boot, Embedded Linux

Build Systems & OS: RHEL, Unix, Linux, Ubuntu, Raspberry Pi OS, FreeRTOS, Poky

Single-board MCUs: PSoC-6, ESP32, i.MX 8M Nano

Miscellaneous: vim, tmux, ssh, scp, grep, regex, Oscilloscopes, Multimeters

PROFESSIONAL EXPERIENCE

Cepheid

Embedded Software Engineer

Sunnyvale, CA

August 2023 – Present, Full-time

- Maintained and enhanced Yocto Linux image to work with changing proprietary hardware and firmware requirements.
- Modified U-Boot stop string code to resemble a typical password prompt to enable intuitive developer/field tech access to U-Boot command line.
- Implemented a secure Chain of Trust across U-Boot and the Linux kernel; configured cryptographic image signing and verification to prevent unauthorized firmware execution.
- Modified Device Tree source files to adjust memory mapping as part of migration to a new SOM.
- Designed and maintained Jenkins CI/CD pipelines to automate the building and archiving of C/C++ firmware, Yocto-based Linux images, and internal engineering tools.
- Engineered a fully automated flashing method for embedded Linux images on System on Modules (SOMs), eliminating manual bottlenecks and accelerating manufacturing processes by over 15x.
- Used the Yocto Project to customize embedded Linux distributions and make them work with our processors and peripherals.
- Wrote robust C/C++ firmware for the GeneXpert modules (medical diagnostic machines); Asymmetric Multi-processing: 1 core FreeRTOS, 4 cores Embedded Linux; Debugging done via GDB over JTAG and/or via log files.
- Rolled out a remote firmware update process that can handle configuration, binaries, and Linux upgrades (implemented the on-device scripts).

Western Digital Corporation

Software Engineer

Milpitas, CA

August 2022 – May 2023, Full-time

- Brought up automated testing rigs that flash firmware builds onto physical SSDs and run functional tests (Pytest) without human intervention.
- Developed, ported, and maintained Python-based Continuous Integration scripts.
- Brought up Hardware-in-the-Loop (HIL) lab systems, integrating Ubuntu bench machines and Raspberry Pis with Jenkins.
- Analyzed/triaged CI test failures every morning and created/updated Jira tickets to streamline development feedback loops.

Two Channel Oscilloscope | *Embedded Systems Design*

April 2021 – June 2021

- Displayed waveforms and frequencies onto an LCD screen; supported vertical and horizontal scaling.
- Read voltage values on 2 GPIO pins using an ADC; data points transferred from ADC buffer to ping-pong buffers via DMA.
- Processing done on one of the ping-pong buffers to calculate frequencies, select data points to render, and detect triggers.
- User input enabled through knobs and command line; knobs are potentiometers whose voltage is monitored by the ADC, commands are sent over UART and parsed on the microcontroller.
- Designed for the PSoC-6 microcontroller and written in C.

Quadruple Software UART | *Embedded Systems Design*

April 2021 – June 2021

- Optimization exercise to implement four UARTs in software and on just the CM4 core.
- UARTs operated concurrently as tasks in FreeRTOS.
- Written in C for the PSoC-6 micro-controller.

Pintos Modification Labs | *Principles of Computer Systems Design*

January 2021 – March 2021

- Improved and implemented aspects of the Pintos operating system, including the ability to block threads, priority-based thread scheduling, and priority donation/inheritance.
- Worked with multi-threading and concurrency primitives.
- Implemented condition variables and locks using the pre-existing semaphore implementation.
- Debugging was performed with GDB.

Prime Number Detector | *Logic Design*

October 2020 – December 2020

- Written in gate level Verilog for the Basys-3 FPGA.
- Determines whether a (16 bit) number is prime; if not, shows the least prime factor.
- Values shown on a 7-segment display.
- Implemented binary long division.